

UNSW, School of Mathematics and Statistics

Profs. Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 08 – Numerical Integration

- 1 Numerical Integration
 - Random variables and expectations
 - Probability density functions
 - Quadrature rules
 - Composite trapezoidal rule
 - Simpson's rule
 - Degree of precision

- Gauss-Legendre rule
 - Non-smooth integrals
 - Change of variables
 - Splitting the interval of integration
- 2 Multi-dimensional integrals
 - Tensor product rules
 - Curse of dimensionality

Random variables and expectations

If X is a random variable with a continuous probability density function $p(x)$, then the probability that X takes a value in the interval $[a, b]$ is

$$\text{Prob}(a \leq X \leq b) = \int_a^b p(x) dx,$$

and the **expected value** of X is

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} xp(x) dx.$$

More generally, the expected value of $f(X)$ is

$$\mathbb{E}[f] = \mathbb{E}[f(X)] = \int_{-\infty}^{\infty} f(x)p(x) dx.$$

Probability density functions

A valid probability density function p must be integrable and satisfy

$$\int_{-\infty}^{\infty} p(x) dx = 1, \quad \text{and} \quad p(x) \geq 0 \text{ for all } x \in \mathbb{R}.$$

Denote the **cumulative** distribution function by

$$P(x) = \text{Prob}(X \leq x) = \int_{-\infty}^x p(\xi) d\xi, \quad P(-\infty) = 0, \quad P(\infty) = 1.$$

Assuming that $P : (-\infty, \infty) \rightarrow (0, 1)$ is differentiable and strictly increasing, the inverse function $P^{-1} : (0, 1) \rightarrow (-\infty, \infty)$ exists. The substitution $y = P(x)$ transforms the expected value of f into an integral over a bounded interval: in fact,

$$dy = P'(x) dx = p(x) dx,$$

so

$$\mathbb{E}[f(X)] = \int_{-\infty}^{\infty} f(x)p(x) dx = \int_0^1 f(P^{-1}(y)) dy.$$

Quadrature rules

A given integral

$$I(f) = \int_a^b f(x) dx$$

can be approximated using a **quadrature rule**:

$$Q_N(f) = \sum_{i=1}^N w_i f(\xi_i),$$

where

- ξ_i are called the **nodes** or **points**, and w_i are called the **weights**.

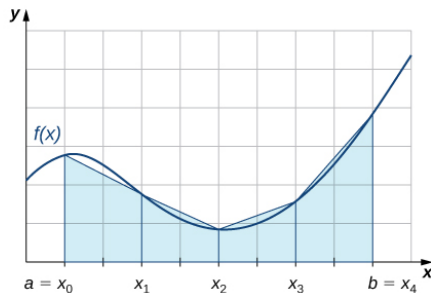
The error in the approximation $I(f) \approx Q_N(f)$, or simply, the **quadrature error**, is

$$E_N(f) = I(f) - Q_N(f).$$

We have different types of quadrature rules.

Composite trapezoidal rule

Graphical illustration: approximate $I(f) = \int_a^b f(x) dx$ using the composite trapezoidal rule.



Basically, we set up a grid

$$a = x_0 < x_1 < x_2 < \cdots < x_N = b$$

and then, approximate f by a linear polynomial interpolant on each subinterval.

Composite trapezoidal rule (continued)

Let h_i be the length of the subinterval $[x_{i-1}, x_i]$, that is,

$$h_i = x_i - x_{i-1}.$$

On each subinterval $[x_{i-1}, x_i]$, define

$$q(x) = \frac{x_i - x}{h_i} f_{i-1} + \frac{x - x_{i-1}}{h_i} f_i, \quad \text{with } f_{i-1} = f(x_{i-1}) \text{ and } f_i = f(x_i),$$

so, q is a linear polynomial interpolates f at the points x_{i-1} and x_i . Hence,

$$I(f) = \sum_{i=1}^N \int_{x_{i-1}}^{x_i} f(x) dx \approx \sum_{i=1}^N \int_{x_{i-1}}^{x_i} q(x) dx = \sum_{i=1}^N \frac{h_i}{2} (f_{i-1} + f_i).$$

Thus, the **composite trapezoidal rule** is defined by

$$I(f) = \int_a^b f(x) dx \approx Q_N^{\text{trap}}(f) := \sum_{i=1}^N \frac{h_i}{2} (f_{i-1} + f_i).$$

In the simplest case when $h_i = h = (b - a)/N$,

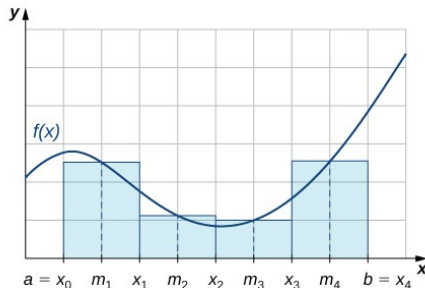
$$Q_N^{\text{trap}}(f) = \frac{h}{2} \sum_{i=1}^N (f_{i-1} + f_i) = \frac{h}{2} f_0 + \sum_{i=1}^{N-1} h f_i + \frac{h}{2} f_N = \sum_{i=0}^N w_i f(x_i)$$

with $w_0 = w_N = h/2$ and $w_i = h$ for $i = 2, 3, \dots, N-1$. The nodes $x_i = a + ih$ for $i = 0, 1, \dots, N$. In a vector form,

$$Q_N^{\text{trap}}(f) = h \left(\frac{1}{2} f_0 + f_1 + f_2 + \cdots + f_{N-1} + \frac{1}{2} f_N \right) = \frac{h}{2} [1, 2, \dots, 2, 1] \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix}.$$

Composite midpoint rule

Graphical illustrations: approximate $I(f)$ using the composite midpoint rule.



The **composite midpoint rule** is defined by

$$I(f) = \int_a^b f(x) dx \approx Q_N^{\text{mid}}(f) := \sum_{i=1}^N h_i f_{i-1/2}, \quad f_{i-1/2} = f(x_{i-1/2}),$$

where $x_{i-1/2} = \frac{1}{2}(x_{i-1} + x_i)$ is the midpoint of the subinterval $[x_{i-1}, x_i]$.

Errors from trapezoidal and midpoint rules

Let $h_i = h = \frac{b-a}{N}$ for all i , and let $M = \max_{x \in [a,b]} |f''(x)|$. Then

$$|E_N^{\text{trap}}(f)| = |I(f) - Q_N^{\text{trap}}(f)| \leq M \frac{b-a}{12} h^2$$

$$|E_N^{\text{mid}}(f)| = |I(f) - Q_N^{\text{mid}}(f)| \leq M \frac{b-a}{24} h^2.$$

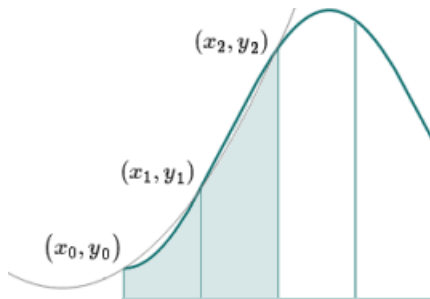
- Therefore, both the composite trapezoidal and the composite midpoint rules are $O(h^2)$ accurate, provided that $f \in \mathcal{C}^2[a, b]$, i.e., the second derivative of f exists and is continuous on $[a, b]$.
- Double the number of subintervals, then

$$|E_{2N}^{\text{trap}}(f)| \approx c_1 \left(\frac{h}{2}\right)^2 = \frac{c_1 h^2}{2^2} \approx \frac{|E_N^{\text{trap}}(f)|}{4}.$$

- To estimate the order of convergence, we use the formula:
 $\log_2(|E_N^{\text{trap}}(f)|/|E_{2N}^{\text{trap}}(f)|).$

Composite Simpson's rule

Graphical illustrations: approximate $I(f)$ using composite Simpson's rule.



- Composite Simpson's rule is based on **quadratic** polynomial approximations and **even number of sub-intervals** N .
- Better accuracy is achieved using the **composite Simpson's rule**.

Composite Simpson's rule (continued)

$$I(f) = \int_a^b f(x) dx = \sum_{i=1,3,5,\dots,N-1} \int_{x_{i-1}}^{x_{i+1}} f(x) dx \approx \sum_{i=1,3,5,\dots,N-1} \int_{x_{i-1}}^{x_{i+1}} q(x) dx,$$

where q is **quadratic** polynomial that interpolates f at x_{i-1} , x_i and x_{i+1}

$$Q_N^{\text{Simp}}(f) = \sum_{i=1,3,5,\dots,N-1} \frac{h_i + h_{i+1}}{6} (f_{i-1} + 4f_i + f_{i+1}).$$

When $h_i = h = (b - a)/N$, and with $\mathbf{w} = [w_i] = \frac{h}{3}[1, 4, 2, 4, 2, \dots, 4, 1]$,

$$Q_N^{\text{Simp}}(f) = \frac{h}{3} (f_0 + 4f_1 + 2f_2 + 4f_3 + 2f_4 + \dots + 4f_{N-1} + f_N) = \sum_{i=0}^N w_i f(x_i).$$

For the absolute error, let $\delta = \max_{x \in [a,b]} |f^{(4)}(x)|$, and then

$$|E_N^{\text{Simp}}(f)| = |I(f) - Q_N^{\text{Simp}}(f)| \leq \delta \frac{b-a}{2880} h^4.$$

Thus, composite Simpson's rule is $O(h^4)$ accurate.

Composite Simpson's rule (continued)

- If we double number of number of uniform subintervals, then

$$|E_{2N}^{\text{Simp}}(f)| \approx c_1 \left(\frac{h}{2}\right)^4 = \frac{c_1 h^4}{2^4} \approx \frac{|E_N^{\text{Simp}}(f)|}{16}.$$

- We can estimate the order of convergence using the following formula: $\log_2(|E_N^{\text{Simp}}(f)|/|E_{2N}^{\text{Simp}}(f)|)$.
- $f \in \mathcal{C}^4[a, b]$ is required. If the fourth derivative of f is unbounded or does not exist on $[a, b]$ then the $O(h^4)$ accuracy is not guaranteed.

Example: Let X be a standard normal random variable and consider evaluating

$$\text{Prob}(0 < X < 1.5) = \int_0^{1.5} \phi(x) dx \quad \text{where} \quad \phi(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}.$$

For both the composite trapezoidal and composite Simpson's rules, compute the absolute errors and convergence order (r) over equal width subintervals $h = (b - a)/N$.

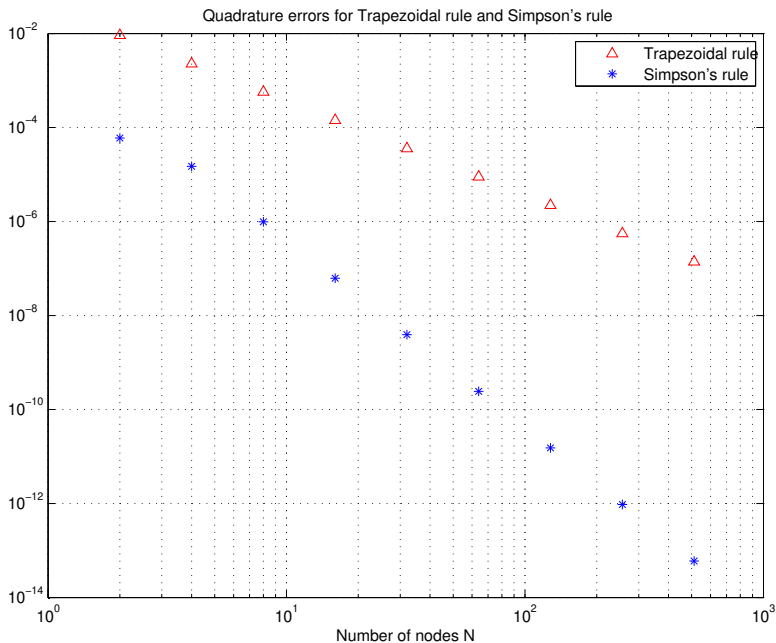
Example (continued)

Note: If the absolute error $|E_N| \approx ch^r$, then by doubling N , $|E_{2N}| \approx c(h/2)^r$. Hence,

$$\frac{|E_N|}{|E_{2N}|} \approx \frac{ch^r}{c(h/2)^r} = 2^r, \quad \text{and so,} \quad r \approx \log_2 \left(\frac{|E_N|}{|E_{2N}|} \right).$$

N	$ E_N^{\text{trap}}(f) $	$\frac{ E_N^{\text{trap}}(f) }{ E_{2N}^{\text{trap}}(f) }$	r	$ E_N^{\text{Simp}}(f) $	$\frac{ E_N^{\text{Simp}}(f) }{ E_{2N}^{\text{Simp}}(f) }$	r
2	9.1673e-03			5.9602e-05		
4	2.2806e-03	4.0196	2.0071	1.4914e-05	3.996	1.9987
8	5.6942e-04	4.0052	2.0019	9.8611e-07	15.825	3.9187
16	1.4231e-04	4.0013	2.0005	6.2315e-08	15.958	3.9941
32	3.5574e-05	4.0003	2.0001	3.9048e-09	15.990	3.9962
64	8.8933e-06	4.0001	2.0000	2.4421e-10	15.997	3.9991
128	2.2233e-06	4.0000	2.0000	1.5266e-11	16.000	3.9997
256	5.5583e-07	4.0000	2.0000	9.5413e-13	16.000	4.0000
512	1.3896e-07	4.0000	2.0000	5.9730e-14	15.974	3.9977

MATLAB script `quadrature.m`, Python file `quadrature.py`.



Degree of precision

Definition

A quadrature rule Q_N has **degree of precision m** if it is exact for all polynomials of degree at most m ; that is,

$$I(p) = Q_N(p) = \sum_{i=1}^N w_i p(\xi_i) \quad \text{for all } p \in \mathbb{P}_m,$$

and $I(p) \neq Q_N(p)$ for at least one $p \in \mathbb{P}_{m+1}$. Here, $\mathbb{P}_m$ denotes the space of polynomials of degree at most m . Recall that w_i and ξ_i are the weights and nodes, respectively.

- If $E_N(f) = I(f) - Q_N(f)$, then the above definition is equivalent to

$$E_N(x^i) = 0 \quad \text{for } i = 0, 1, \dots, m, \quad \text{but } E_N(x^{m+1}) \neq 0.$$

Example

Verify the degree of precision m for each of the following quadrature rules.

① Composite midpoint and trapezoidal rules: $m = 1$.

② Composite Simpson's rule: $m = 3$.

Solution. For simplicity, assume that $h_i = h = (b - a)/N$, so $x_i = a + ih$ for $i = 0, 1, \dots, N$.

Composite midpoint rule:

$$Q_N^{\text{mid}}(1) = \sum_{i=1}^N h = b - a = I(1).$$

$$\begin{aligned} Q_N^{\text{mid}}(x) &= \sum_{i=1}^N h x_{i-\frac{1}{2}} = \sum_{i=1}^N (x_i - x_{i-1})(x_i + x_{i-1})/2 = \sum_{i=1}^N (x_i^2 - x_{i-1}^2)/2 \\ &= \sum_{i=1}^N \int_{x_{i-1}}^{x_i} x \, dx = \int_a^b x \, dx = I(x). \end{aligned}$$

$$N = 1, \quad Q_N^{\text{mid}}(x^2) = (b-a)x_{\frac{1}{2}}^2 = (b-a)\left(\frac{a+b}{2}\right)^2 \neq \frac{b^3 - a^3}{3} = \int_a^b x^2 \, dx = I(x^2).$$

Composite trapezoidal rule:

$$Q_N^{\text{trap}}(1) = \sum_{i=1}^N \frac{h}{2}(1 + 1) = \sum_{i=1}^N h = Nh = b - a = I(1).$$

$$Q_N^{\text{trap}}(x) = \sum_{i=1}^N \frac{h}{2}(x_i + x_{i-1}) = \sum_{i=1}^N \frac{x_i^2 - x_{i-1}^2}{2} = \int_a^b x \, dx = I(x).$$

If $N = 1$,

$$Q_N^{\text{trap}}(x^2) = \frac{b-a}{2}(b^2 + a^2) \neq \frac{b^3 - a^3}{3} = \int_a^b x^2 \, dx = I(x^2).$$

Composite Simpson's rule: For the degree of precision, it is enough to consider one panel, i.e. $N = 2$. The affine substitution

$$x = \frac{a+b}{2} + \frac{b-a}{2}t = \frac{1}{2}[(1-t)a + (1+t)b]$$

maps $t \in [-1, 1]$ to $x \in [a, b]$. Thus it suffices to treat the special case $[a, b] = [-1, 1]$. In this case, $h = (b-a)/2 = 1$, and

$$Q_2^{\text{Simp}}(f) = \frac{b-a}{6} (f(a) + 4f((a+b)/2) + f(b)) = \frac{1}{3}(f(-1) + 4f(0) + f(1)).$$

Therefore,

$$Q_2^{\text{Simp}}(x^k) = \int_{-1}^1 x^k dx \quad \text{for } k = 0, 1, 2, 3,$$

but

$$Q_2^{\text{Simp}}(x^4) = \frac{1}{3}(1 + 4 \cdot 0 + 1) = \frac{2}{3} \neq \int_{-1}^1 x^4 dx = \frac{2}{5}.$$

Hence, the degree of precision is 3.

N -point Gauss–Legendre quadrature rule

The N -point **Gauss–Legendre rule** for approximating $\int_{-1}^1 f(x) dx$ takes the form:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^N w_i f(\xi_i), \quad E_N(f) = \int_{-1}^1 f(x) dx - \sum_{i=1}^N w_i f(\xi_i).$$

We compute the unknown weights $w_1, w_2, \dots, w_N$ and the unknown nodes $\xi_1, \xi_2, \dots, \xi_N$ using the $2N$ conditions:

$$E_N(1) = E_N(x) = E_N(x^2) = \dots = E_N(x^{2N-1}) = 0,$$

so the degree of precision is $2N - 1$.

The M-file **gauleg.m** / Python function **gauss_legendre.py** computes the weights and nodes of a Gauss–Legendre rule for approximating $\int_a^b f(x) dx$.

N -point Gauss–Legendre quadrature rule (continued)

Example

Find the weights and the nodes of the 2-point Gauss–Legendre rule,

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^2 w_i f(\xi_i).$$

Solution. 2-point Gauss–Legendre rule has degree of precision 3. Thus,

$$\begin{cases} w_1 + w_2 = \int_{-1}^1 dx = 2 \\ w_1 \xi_1 + w_2 \xi_2 = \int_{-1}^1 x dx = 0 \\ w_1 \xi_1^2 + w_2 \xi_2^2 = \int_{-1}^1 x^2 dx = \frac{2}{3} \\ w_1 \xi_1^3 + w_2 \xi_2^3 = \int_{-1}^1 x^3 dx = 0. \end{cases}$$

Therefore, $w_1 = w_2 = 1$, $\xi_1 = -1/\sqrt{3}$ and $\xi_2 = 1/\sqrt{3}$. This implies

$$\int_{-1}^1 f(x) dx \approx f(-\eta) + f(\eta) \quad \text{with} \quad \eta = \frac{1}{\sqrt{3}}, \quad \text{has degree of precision 3.}$$

N -point Gauss-Legendre quadrature rule (continued)

Example

With $w = 5/9$ and $\eta = \sqrt{3/5}$, show that the 3-point Gauss-Legendre rule,

$$I(f) = \int_{-1}^1 f(x) dx \approx Q_3(f) := w f(-\eta) + (2 - 2w)f(0) + w f(\eta),$$

has degree of precision $m = 5$.

Solution. For $m = 1, 3, 5$,

$$I(x^m) = 0 = Q_3(x^m).$$

For the other cases,

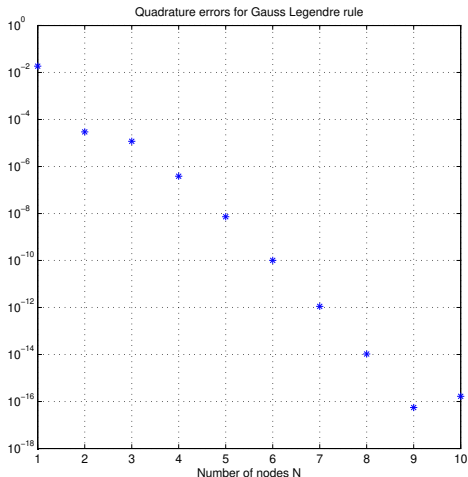
$$Q_3(f) = \begin{cases} w + (2 - 2w) + w = \int_{-1}^1 dx = I(1) & \text{if } f(x) = 1, \\ 2w\eta^2 = 2/3 = \int_{-1}^1 x^2 dx = I(x^2) & \text{if } f(x) = x^2, \\ 2w\eta^4 = \frac{2}{5} = \int_{-1}^1 x^4 dx = I(x^4) & \text{if } f(x) = x^4, \\ 2w\eta^6 = \frac{6}{25} \neq \int_{-1}^1 x^6 dx = I(x^6) = \frac{2}{7} & \text{if } f(x) = x^6. \end{cases}$$

N -point Gauss–Legendre quadrature rule (continued)

Example

Use N -point Gauss-Legendre rule to estimate $\int_0^{1.5} \phi(x) dx$

N	$ E_N(f) $
1	1.8513e-02
2	2.9616e-05
3	1.1864e-05
4	3.8498e-07
5	7.2948e-09
6	1.0115e-10
7	1.1166e-12
8	1.0547e-14
9	5.5511e-17
10	1.6653e-16



Non-smooth integrands

The preceding quadrature rules are less accurate if applied to a function that is not smooth. See for example the table below for approximating $\int_0^1 \sqrt{x} dx$.

N	$ E_N^{\text{trap}}(f) $	r	$ E_N^{\text{mid}}(f) $	r	$ E_N^{\text{Simp}}(f) $	r
2	6.31e-02		1.63e-02		2.86e-02	
4	2.34e-02	1.432	6.31e-03	1.373	1.01e-02	1.496
8	8.54e-03	1.454	2.37e-03	1.416	3.59e-03	1.499
16	3.09e-03	1.468	8.70e-04	1.443	1.27e-03	1.500
32	1.11e-03	1.478	3.16e-04	1.461	4.48e-04	1.500
64	3.96e-04	1.485	1.14e-04	1.473	1.59e-04	1.500

Note: MATLAB ([quad](#)) and Python ([scipy.integrate.quad](#)) provide some **adaptive** quadrature routines that automatically select points and weights to achieve a specified error tolerance.

Change of variables

Sometimes a change of variable can help when evaluating an integral of a non-smooth function.

For instance, we expect poor accuracy with

$$\int_0^{\pi/2} \sqrt{\sin x} \, dx, \quad (\sqrt{\sin x})' = \frac{\cos x}{2\sqrt{\sin x}} \rightarrow \infty \text{ as } x \rightarrow 0^+,$$

(the integrand has an unbounded derivative at $x = 0$).

However, the substitution $t = \sqrt{x}$ (then $x = t^2$) gives

$$\int_0^{\sqrt{\pi/2}} 2t\sqrt{\sin t^2} \, dt, \quad (t\sqrt{\sin t^2})' = \sqrt{\sin t^2} + \frac{t^2 \cos t^2}{\sqrt{\sin t^2}} \rightarrow 0 \text{ as } t \rightarrow 0^+,$$

so the derivative remains bounded. This substitution is motivated by the fact that $\sqrt{\sin x} \approx \sqrt{x}$ as $x \rightarrow 0^+$.

Splitting the interval of integration

Sometimes it can help to split the interval of integration into two or more subintervals, each of which is an easier integral.

We expect poor accuracy with

$$\int_0^4 |\cos x| dx,$$

because the derivative of the integrand is discontinuous at $x = \frac{\pi}{2}$. However, as $\cos x \geq 0$ for $x \in [0, \frac{\pi}{2}]$ and $\cos x \leq 0$ for $x \in [\frac{\pi}{2}, 4]$,

$$\int_0^4 |\cos x| dx = \int_0^{\frac{\pi}{2}} \cos x dx + \int_{\frac{\pi}{2}}^4 (-\cos x) dx.$$

Both sub-integrals have smooth integrands.

Tensor-product rules – a two-dimensional extension

Given a 1-dimensional quadrature rule

$$\int_0^1 f(t) dt \approx \sum_{i=1}^n w_i f(\xi_i).$$

Construct a 2-dimensional **tensor-product rule**:

$$\begin{aligned} \int_0^1 \underbrace{\int_0^1 f(x, y) dy}_{=g(x)} dx &\approx \sum_{j=1}^n w_j \underbrace{\int_0^1 f(\xi_j, y) dy}_{=g(\xi_j)} \\ &\approx \sum_{j=1}^n w_j \sum_{k=1}^n w_k f(\xi_j, \xi_k) = \sum_{j,k=1}^n w_j w_k f(\xi_j, \xi_k). \end{aligned}$$

MATLAB **tensor2.m**, Python **tensor2d** in **tensor.py**

Consider a tensor-product Simpson rule, with N even and $h = 1/N$. Let $\mathbf{w} = [w_j] = \frac{h}{3}[1, 4, 2, 4, 2, \dots, 4, 1] \in \mathbb{R}^{N+1}$,

$$I(f) = \int_0^1 \int_0^1 f(x, y) dy dx \approx Q_N^{\text{Simp}}(f) = \sum_{j,k=0}^N w_j w_k f(x_j, y_k),$$

$x_j = jh$ and $y_k = kh$. Assuming that f is smooth, we have

$$|I(f) - Q_N^{\text{Simp}}(f)| \leq ch^4,$$

for some positive constant c .

Generalizations

- Different 1-dimensional rules in each variable:
MATLAB `tensor2d.m`; Python `tensor2d` in `tensor.py`
- A d -fold tensor product of an $(N+1)$ -point one-dimensional rule gives a d -dimensional rule with $(N+1)^d$ points.
MATLAB `tensor3.m`; Python `tensor3` in `tensor.py`

Curse of dimensionality

Example (The curse of dimensionality)

How long would it take to evaluate an integral over $[0, 1]^d$ using a d -fold tensor-product composite Simpson rule, with $d = 50$, if we run our program on a 3GHz quad-core PC and want an error less than 0.01?

- Speed of the computer = $4 \times 3 \times 10^9 = 1.2 \times 10^{10}$ clock cycles/sec.
- Assume that the composite Simpson rule error is bounded by h^4 .
- To achieve error less than 0.01, we need $h^4 \leq 0.01$, that is, $N^4 \geq 100$, so $N = 4$ in each dimension. Then the total number of nodes is 5^{50} .
- Assume that we need one clock cycles per evaluating each term.
Then the total CPU time in **years** is approximately

$$\frac{5^{50}}{1.2 \times 10^{10} \times 60 \times 60 \times 24 \times 365} \approx 2.35 \times 10^{17}.$$

- Obviously, we need a faster method.